Performance Benchmark Report

CSTAM 3.0 challenge "The Resilient IP Optimizer" (FelCloud) - Phase 2 deliverable
Repository: https://github.com/CSTAM-well-contained/monorepo

Benchmark report

All numbers are measured, not estimated. Each line names the command that reproduces it and the lab notebook entry with the details (experiments.md).

Where: a laptop (WSL2 Debian 13, Docker 29) running real HAProxy 3.2, keepalived 2.3, PostgreSQL 18 and our binaries, on a Docker bridge with the FelCloud IP plan (10.20.0.0/24); and **FelCloud** for the VM driver (B4) and network isolation (B9). FelCloud has no free public IP yet (`ExternalIpAddressExhausted`), so the gateways cannot be reached from the internet there; on OVN, failover will add the virtual-port rebinding time (expected 10-300 ms), to be measured once the public IPs exist.

Brief checklist (Phase 2 deliverable): provisioning latency B4, B5 · gateway failover time B6, B8 · IP recycling speed B7 · load test metrics B8 · isolation B8, B9 · reload and rollback B1. Phase 2 validation summary: phase2.md.

Summary

#	Metric	Result	Target
B1	Requests dropped by hot reloads	**0 of 96,606** over 20 reloads; 0 of 40,667 in a later run	zero downtime
B1	Broken config: detection + rollback	rejected by `haproxy -c` before touching HAProxy; a config that loads but fails verification is rolled back in **73-142 ms**, 0.72-0.97 s end to end	sub-second
B2	New team routed after cpd publishes it	**24 ms** (runtime API, no reload)	
B3	1000 parallel allocations, 128 slots	128 allocated, 872 clean `pool exhausted`, **no IP or slot given twice**	no conflicts
B4	FelCloud VM sandbox create / delete	**23 s** to ACTIVE, **14 s** to delete server + port	
B5	50 simultaneous registrations	**50/50 routed, p50 3.4 s, p95 4.9 s**, 0 errors, 50 distinct IPs and slots	10+ teams, error-free
B6	Planned failover from the API / dashboard	VIP on the peer **0.30-0.31 s** after the call, 1 probe of ~175 lost (20 ms probes)	manual failover
B1	Broken config pushed through the API, both gateways	valid-but-broken version rolled back in **139 ms** (0.80 s total), syntax error rejected in **23 ms**, **0 failed requests** during the sequence	auto-rollback
B6	Gateway failover (takeover through the VIP)	**1.3-1.9 s** frozen master, 0.2 s clean stop; HAProxy killed: **0.45-0.61 s** (systemd restarts it, no failover needed)	seamless
B7	Teardown of 50 teams	sandboxes and routes gone in **7 s**; IPs back in the pool at **63 s** (60 s quarantine by design)	immediate release

#	Metric	Result	Target
B8	HTTP load through the VIP, 20 teams	**3,000 req/s, 0 errors, 0 misrouted, p99 1.2-1.6 ms** (new connection per request: p99 4.5 ms)	load test metrics
B8	Failover **under that load**	longest time without any successful answer: frozen master **1.51 s**; HAProxy killed **1.9 s** (plain), **0.46-0.69 s** (systemd); 0 misrouted	seamless
B8	One team flooded (~79,000 req/s)	99.9 % answered 429; the 19 other teams: **0 errors, p99 3.7 ms**	isolation
B5	Rerun: 20 and 50 at once	20/20 p95 3.1 s; 50/50 p95 7.0 s with 24 teams already up; 0 errors	10+ teams, error-free
B7	Lease expiry, nobody acting	at the expiry second: route removed (404), IP quarantined, sandbox deleted 1 s later	automated teardown
B9	Network isolation **on FelCloud** (lab VMs)	team to team, team to gateway management and to cp: **all dropped**; gateway to team: open	network isolation

Reproduce everything

```
make build
deploy/dev/demo.sh up                     # GW_MODE=systemd for the systemd gateways (L8)
deploy/dev/demo.sh teams 20
deploy/dev/demo.sh load 20 64             # B8 steady load
TIMEOUT=0.3s deploy/dev/demo.sh load 12 64 pause   # B8 failover under load (or kill)
CPD_WORKERS=16 HACK=s50 deploy/dev/demo.sh stress 50   # B5
deploy/dev/demo.sh failover pause         # B6
deploy/dev/demo.sh teardown               # B7
deploy/dev/demo.sh reload                 # B1
```

B1 - Zero-downtime reload and rollback

`go test -tags integration -v -run 'TestBrokenConfigRollsBack|TestReloadDropsNoRequest' ./internal/gw/applier/`
(or `deploy/dev/demo.sh reload`). Notebook L1, L6.

- 20 reloads under continuous load: 96,606 requests, 0 failed. Slowest apply 356 ms.
- Broken config that passes `haproxy -c` but fails the post-reload verification: check 151 ms, reload 175 ms, verify timeout 501 ms, **rollback 142 ms**, total 0.97 s. (L1: rollback 73 ms, total 0.72 s.) A config with a syntax error never reaches HAProxy.

Through cpd on the running gateways (`deploy/dev/demo.sh config`, notebook L11), each gateway:

Version	Change	Outcome	check	reload	verify	rollback	total
c1	rate limit 1000	applied	45 ms	118 ms	2 ms	-	165 ms
c2	valid syntax, health check broken	rolled back	43 ms	119 ms	500 ms (timeout)	**139 ms**	0.80 s
c3	unknown keyword	rejected by `haproxy -c`	23 ms	-	-	-	23 ms
c4	manual rollback to c1 (new version)	applied	39 ms	117 ms	2 ms	-	158 ms

A team was requested every 20 ms through the VIP during the whole sequence: 316 x 200, 0 errors.

B2 - Route propagation

`go test -tags integration -run TestAgentDrivesRealHAProxy -v ./internal/gw/agent/` (L5).
cpd publishes generation 2 with a new team; the team answers through HAProxy 24 ms later.

B3 - IPAM under contention

`go test -race -tags integration -v ./internal/ipam/` (L3). 1000 concurrent "create team + allocate" transactions on 128 slots: 128 allocated, 872 `ErrPoolExhausted`, no duplicate, 3.8 s total with the race detector. With 999 slots: the 159 pool IPs minus the reserved .68.

B4 - OpenStack provisioning (FelCloud)

`OS_CLOUD=cstam go test -tags e2e -run TestE2ESandbox -v ./internal/openstack/` (L4).
One `G0.basic.1c1g` Debian 12 VM on a Neutron port with a fixed IP: ACTIVE in 23 s, server and port deleted in 14 s. The instance quota (10 for the project, 3 used by the platform) limits a FelCloud stress test to about 6 VM teams; more teams need the shared-host driver (Phase 2 design).

B5 - Concurrent provisioning stress test

`CPD_WORKERS=16 deploy/dev/demo.sh up; HACK=s50 deploy/dev/demo.sh stress 50` (L7).
All registrations are sent at the same time; each team is "done" when its own page (team name and IP) answers through the VIP.

Teams	cpd workers	API answers (all)	Routed	p50	p95	max	Distinct IPs / slots
20	4	0.32 s	20/20	2.37 s	4.02 s	4.16 s	20 / 20
20	16	0.40 s	20/20	2.11 s	2.42 s	3.55 s	20 / 20
50	16	0.51 s	50/50	3.39 s	4.93 s	5.10 s	50 / 50
20 (rerun, L15)	4	0.25 s	20/20	1.88 s	3.08 s	3.09 s	20 / 20
50 (rerun, 24 teams already up)	4	0.33 s	50/50	3.95 s	6.97 s	7.01 s	50 / 50

The API answers every registration in under 0.6 s (IP and slot are allocated in that call); the rest is sandbox creation, bounded by `-workers`. Route propagation itself is milliseconds (B2).

B6 - Gateway failover

`deploy/dev/demo.sh failover pause|kill`: a probe requests a team through the VIP every 20 ms (200 ms timeout) while the MASTER is broken (L2, L6).

Failure	Takeover (6+ runs)	Mechanism
keepalived stopped cleanly	0.20 s	priority-0 advert
HAProxy killed	1.5, 1.57, 1.86, 2.31 s	`chk_gateway` fails twice (interval 1 s)
master frozen (`docker pause`)	1.32, 1.58, 1.62, 1.7, 1.83, 1.85 s	3 x 0.5 s + skew = 1.8 s in theory

Failure	Takeover (6+ runs)	Mechanism
HAProxy killed, gateway under systemd (`GW_MODE=systemd`, L8)	VIP down 0.45, 0.46, 0.59, 0.61 s, no failover	`Restart=always` restarts HAProxy before `chk_gateway` fails twice

On the VMs, `vrrp_track_process` should bring the HAProxy-killed case under 1 s (cannot be measured in containers).

Planned failover (`POST /v1/gateways/{gw}/failover`, the dashboard button; `demo.sh failover planned`, notebook L10): the agent writes keepalived's force-backup file, the MASTER goes to FAULT and sends a priority-0 advert. Twice: peer serving **304 ms and 310 ms** after the API call, 1 of ~175 probes lost each time. Refused (409) when the peer is not alive, in sync and healthy.

B7 - Teardown and IP recycling

`deploy/dev/demo.sh teardown` after B5 (L6, L7). 50 teams: routes removed at once, all sandboxes deleted in 7 s; IPs stay in quarantine until both gateways acked the route removal and 60 s passed, then all 50 are free at 63 s. The next hackathon received 10.20.0.32 again.
The 60 s quarantine is a deliberate safety margin (`ipam.Quarantine`, for ARP caches and keep-alive connections); the gateway-ack condition alone is met within milliseconds.
With `cpd -quarantine 10s` (video) the 20 IPs of a hackathon were free 11 s after its end.

Rerun (L15): 50 teams deleted in 6.9 s, IPs free at 63.0 s, 0 routes left, a deleted team's subdomain answers 404. **Lease expiry** with nobody acting (hackathon with a 30 s lease, 3 teams): at the expiry second the routes were removed (200 -> 404) and the IPs quarantined; the teams were deleted 1 s later.

B8 - HTTP load test through the gateways

`deploy/dev/demo.sh load [SECONDS] [WORKERS] [pause|kill]`, which runs `cstamctl load` (notebook L9). The load tool sends `GET /` to the VIP with each team's `Host` in turn, at a fixed total rate (`RATE`, default 150 req/s per team, under the 200 req/s per-team limit), and checks that every 200 is **the page of the requested team** (a wrong team's page counts as misrouted). With `pause` or `kill` the MASTER is broken 5 s into the run. "Longest time without a successful answer" is the outage a client sees, measured from the answers themselves, so it also counts requests that hang on a frozen gateway (with `TIMEOUT=0.3s`, like a client that retries quickly).

Steady load, 20 teams, 64 workers, 20 s, laptop:

Connections	Requests	Errors / misrouted	Rate	p50	p90	p99	max
keep-alive	60,000	0 / 0	3,000/s	0.4 ms	0.9 ms	1.6 ms	6.9 ms
keep-alive, systemd gateways	29,999	0 / 0	3,000/s	0.4 ms	0.5 ms	1.2 ms	8.4 ms
new connection per request	59,989	0 / 0	2,999/s	1.7 ms	2.6 ms	4.5 ms	75.7 ms

Failover while serving 3,000 req/s (64 workers, 0.3 s request timeout unless noted):

Failure	Gateways	Longest time without success	Failed requests	Misrouted
MASTER frozen (`docker pause`)	plain	1.51, 1.51, 1.51 s	320 (timeouts)	0

Failure	Gateways	Longest time without success	Failed requests	Misrouted
MASTER frozen, 2 s timeout	plain	2.0 s (bounded by the client timeout)	64	0
HAProxy killed	plain	1.91 s (2 s timeout: 1.72-1.91 s)	~4,800 (refused connections)	0
HAProxy killed	systemd (`Restart=always`)	**0.46, 0.55, 0.69 s**, no failover	1,183-1,868	0

Rate limit and isolation: one client floods team01 with 32 workers and no rate (about 79,000 req/s, 1.59 M requests in 20 s) while 19 other teams receive 2,850 req/s together:

Traffic	Result
Flooded team	1,968 answered (the first 10 s budget), 1,584,829 refused with 429
19 other teams	42,747 requests, **0 errors**, p50 0.6 ms, p99 3.7 ms, max 15.5 ms

Capacity note: the gateway itself answered ~95,000 req/s of 429s on the laptop; served traffic per team is capped on purpose at 200 req/s (`-rate-limit 2000` per 10 s), so the aggregate ceiling grows with the number of teams. The counters are per gateway: after a failover a flooding client starts from zero on the new MASTER (security.md, known gaps).

B9 - Network isolation on FelCloud

Lab VMs on the real Neutron, results on their noVNC screen (phase2.md, L14). From a team port every internal target times out (dropped), including the control plane's SSH after the `cstam-team` egress fix; from a gateway-group port the team page answers.

Phase 2 validation: integration and testing (30 points)

How each Phase 2 requirement of the brief was validated, with the command that reproduces it and the measured result. Run on the local stack (real HAProxy 3.2, keepalived, PostgreSQL 18, our binaries, FelCloud IP plan on a Docker bridge) and on **FelCloud itself** for network isolation. Details per run in the lab notebook (L14, L15).

Requirement	Points	Verdict
Concurrent provisioning stress test, 10+ simultaneous teams	15	**pass**: 20/20 and 50/50, 0 errors
Automated teardown and lifecycle security	10	**pass**: expiry and hackathon end purge sandboxes, routes and IPs with nobody involved
Network isolation and access safeguards	5	**pass on FelCloud**: team-to-team and team-to-management traffic dropped by Neutron; gateways still reach teams
Deliverable: benchmark report		benchmark.md (B1-B9)
Deliverable: API integration and security architecture docs		api.md, openapi.yaml, security.md

1. Concurrent provisioning stress test (15)

`deploy/dev/demo.sh stress N` sends N team registrations to the API at the same time, then waits until each team answers **through the VIP with its own page**: sandbox created, private IP leased, route published on both gateways.

N at once	API answers	Routed	Rejected	Distinct IPs / slots	Call to own page through the VIP
20	all in 254 ms	**20/20**	0	20 / 20	p50 1.9 s, p95 3.1 s, max 3.1 s
50 (with 24 teams already running)	all in 328 ms	**50/50**	0	50 / 50	p50 4.0 s, p95 7.0 s, max 7.0 s

After each run: both gateways acknowledged the same routes generation (82), `/v1/status` had no alert. Uniqueness under contention is also proven at the database level: 1000 parallel allocations on 128 slots never gave an IP or slot twice (benchmark B3).
On FelCloud a sandbox is a VM (23 s to ACTIVE, B4); the project quota (10 instances) limits a VM run to about 6 teams, which is why the 10+ run uses the container driver: same API, controller, IPAM and gateways.

2. Automated teardown and lifecycle security (10)

Trigger	Command	Result
Hackathon ends	`HACK=stress50 deploy/dev/demo.sh teardown`	50 sandboxes deleted in **6.9 s**; routes removed first; the 50 IPs in quarantine, all **free at 63 s**; 0 routes left, a deleted team's subdomain answers **404**, 0 leftover containers

Trigger	Command	Result
Lease expires (nobody acts)	hackathon with `lease_ttl_seconds: 30`, 3 teams	at the expiry second: IPs quarantined, routes removed (200 -> **404**), teams deleted 1 s later
Failed provisioning	(tests)	failed team cleaned after 10 min, its sandbox deleted (`TestCreateFailureCleansUp`)

Why it is safe ("sanitizing IP allocations"): an IP goes back to the pool only when (1) its route was removed in the same transaction that released it, (2) **every live gateway acknowledged** a routes generation without it, (3) 60 s of quarantine passed, (4) the cloud says no port still holds it. So no gateway can still send a user to a recycled IP (`TestGatewayAckGatesRecycling`, `TestIPTakenStaysQuarantined`). Every step is an event in `/v1/events` and on the dashboard.

3. Network isolation and access safeguards (5)

On FelCloud, without floating IPs. Three throwaway VMs `cstam-lab-iso-*` (lab range .192-.194): **A** and **B** in `cstam-team` like any team sandbox (B serves a page on port 80), **C** in `cstam-gw` like a gateway. At boot, A and C try a TCP connection to each target three times and show the result on their screen, read through the noVNC console.
OPEN = connected, REFUSED = reachable but nothing listens, FILTERED = dropped by Neutron.

From \ to	team B :80 / :8000 / :22	gw :22, :8404 (management)	gw :80 (public entry)	cp :22 (SSH)	cp :8443 (API)	cp :5432 (DB)
team A, before the fix	FILTERED / FILTERED / FILTERED	FILTERED	REFUSED	**OPEN**	FILTERED	FILTERED
team A, after the fix	FILTERED / FILTERED / FILTERED	FILTERED	FILTERED	**FILTERED**	FILTERED	FILTERED
gateway-like C (control)	**OPEN** / REFUSED / FILTERED	FILTERED	REFUSED	OPEN	REFUSED	FILTERED

- Teams cannot reach each other on any port; the gateway group can reach team app ports, so the FILTERED results are the security group at work, not a dead VM.
- Gateway management (SSH, stats) only answers the control plane; `cpd` only the agents.
- **Found and fixed:** the control plane's SSH is open to admins from anywhere (key-only), so a sandbox could reach it. Security groups can only allow, so `cstam-team` now has **no default egress**: it may open connections to anything outside 10.20.0.0/24 (internet, metadata service) and nothing inside it (`infra/pulumi/egress.go`, 24 prefixes computed from the IP plan, unit-tested). Replies to the gateways need no rule (stateful). Re-measured: everything internal FILTERED from a team, the gateway still reaches the team page.

Team VM A after the fix (FelCloud noVNC console): every internal target dropped.

```
CSTAMPROBE from cstam-lab-iso-a 10.20.0.192  after 3 rounds, 19:59:47Z
10.20.0.193:80     FILTERED
10.20.0.193:8000   FILTERED
10.20.0.193:22     FILTERED
10.20.0.11:22      FILTERED
10.20.0.11:8404    FILTERED
10.20.0.11:80      FILTERED
10.20.0.12:22      FILTERED
10.20.0.5:22       FILTERED
10.20.0.5:8443     FILTERED
10.20.0.5:5432     FILTERED

cstam-lab-iso-a login:
```

Gateway-like VM C: the team page (10.20.0.193:80) answers, management ports stay closed.

```
CSTAMPROBE from cstam-lab-iso-c 10.20.0.194  after 3 rounds, 19:59:16Z
10.20.0.193:80     OPEN
10.20.0.193:8000   REFUSED
10.20.0.193:22     FILTERED
10.20.0.11:22      FILTERED
10.20.0.11:8404    FILTERED
10.20.0.11:80      REFUSED
10.20.0.12:22      FILTERED
10.20.0.5:22       OPEN
10.20.0.5:8443     REFUSED
10.20.0.5:5432     FILTERED

cstam-lab-iso-c login:
```

Team VM A before the fix: the control plane's SSH (10.20.0.5:22) was reachable.

```
CSTAMPROBE from cstam-lab-iso-a 10.20.0.192  after 3 rounds, 18:51:18Z
10.20.0.193:80     FILTERED
10.20.0.193:8000   FILTERED
10.20.0.193:22     FILTERED
10.20.0.11:22      FILTERED
10.20.0.11:8404    FILTERED
10.20.0.11:80      REFUSED
10.20.0.12:22      FILTERED
10.20.0.5:22       OPEN
10.20.0.5:8443     FILTERED
10.20.0.5:5432     FILTERED

cstam-lab-iso-a login:
```

Also verified on real HAProxy (`go test -tags integration -run TestGatewayGuards ./internal/gw/applier/`): `/__gw/*` (health and version) is a silent 404 from outside addresses, a flooded team gets 429 while another team keeps answering, and `A.HACKA.TEST:80` does not bypass the per-team limit. Under load: one team at ~79,000 req/s, the 19 others 0 errors, p99 3.7 ms (benchmark B8).

Reproduce

```
make build && deploy/dev/demo.sh up
HACK=stress20 deploy/dev/demo.sh stress 20
HACK=stress50 deploy/dev/demo.sh stress 50
HACK=stress50 deploy/dev/demo.sh teardown
go test -count=1 -tags integration -run TestGatewayGuards ./internal/gw/applier/
# FelCloud isolation lab: notebook L14 (cloud-init files, commands, cleanup)
```

What is left for the FelCloud run (needs public IPs from FelCloud)

The same checks through the real public entry once floating IPs exist: a team page through `*.cstam.felcloud.tn`, failover with OVN, VM-based stress up to the quota. Isolation itself does not depend on them and is already measured above.